

Laboratorium Ilmu Rekayasa dan Komputasi

TEKNIK INFORMATIKA
STEI ITB

IF2224 - Formal Language and
Automata Theory

Arion Compiler

Milestone 2 - Syntax Analysis

Milestone 2

Syntax Analysis

Release : Kamis, 16 April 2026

Deadline : Kamis, 7 Mei 2026 pukul 23.59 WIB

Revisi : -

Daftar Revisi

[DD-MM-YY] Revisi X. Isi Revisi

Daftar Isi

Daftar Revisi.....	2
Daftar Isi.....	4
I. Teori Singkat.....	5
II. Spesifikasi Tugas Besar.....	8
A. Revisi Program Milestone 1.....	8
B. Spesifikasi Program.....	8
C. Daftar Node.....	9
D. Laporan.....	19
III. Teknis Pengerjaan.....	20
IV. Pengumpulan.....	21
V. Penilaian.....	22
Referensi.....	24
Lampiran A. Link Penting.....	25
Lampiran B. Syntax Diagram.....	26

I. Teori Singkat

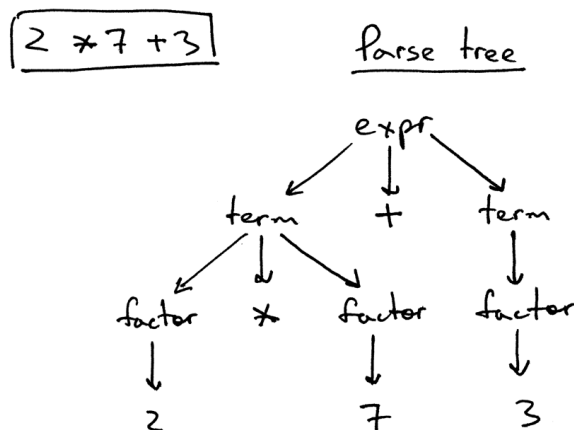
Pada milestone 1 lexical analysis, source code sudah dibentuk menjadi token-token tertentu. Token-token ini akan digunakan oleh tahap selanjutnya yaitu Syntax Analysis atau sering disebut parser.

Syntax Analysis atau **Parser** bertugas untuk memeriksa urutan token yang dihasilkan oleh lexical analyzer agar sesuai dengan **aturan tata bahasa (grammar)** dari bahasa pemrograman yang dikompilasi.

Parser akan membangun struktur sintaks seperti **Parse Tree** atau AST (Abstract Syntax Tree) yang merepresentasikan struktur hierarkis program sumber berdasarkan grammar yang telah didefinisikan.

Tahapan Parser bertujuan untuk:

- Memastikan urutan token membentuk **struktur sintaks yang valid**.
- Mendeteksi dan melaporkan **kesalahan sintaks** (syntax error).
- Mempersiapkan representasi program untuk tahap berikutnya.

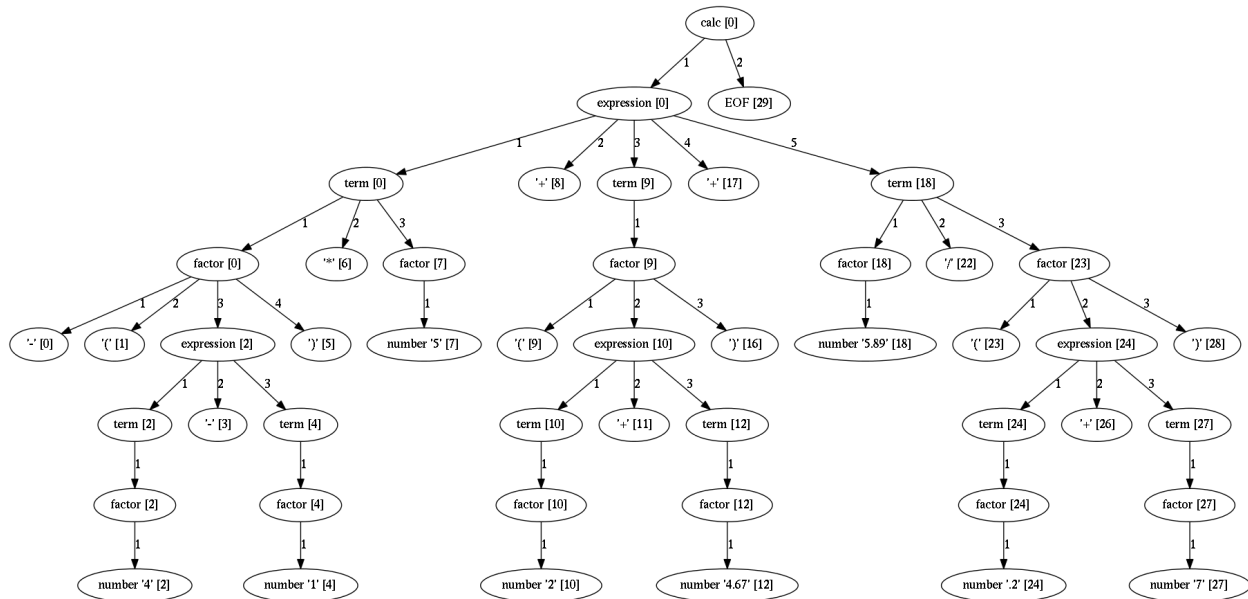


Gambar Parse Tree

(Sumber: <https://ruslanspivak.com/lbasi-part7>)

Parse Tree

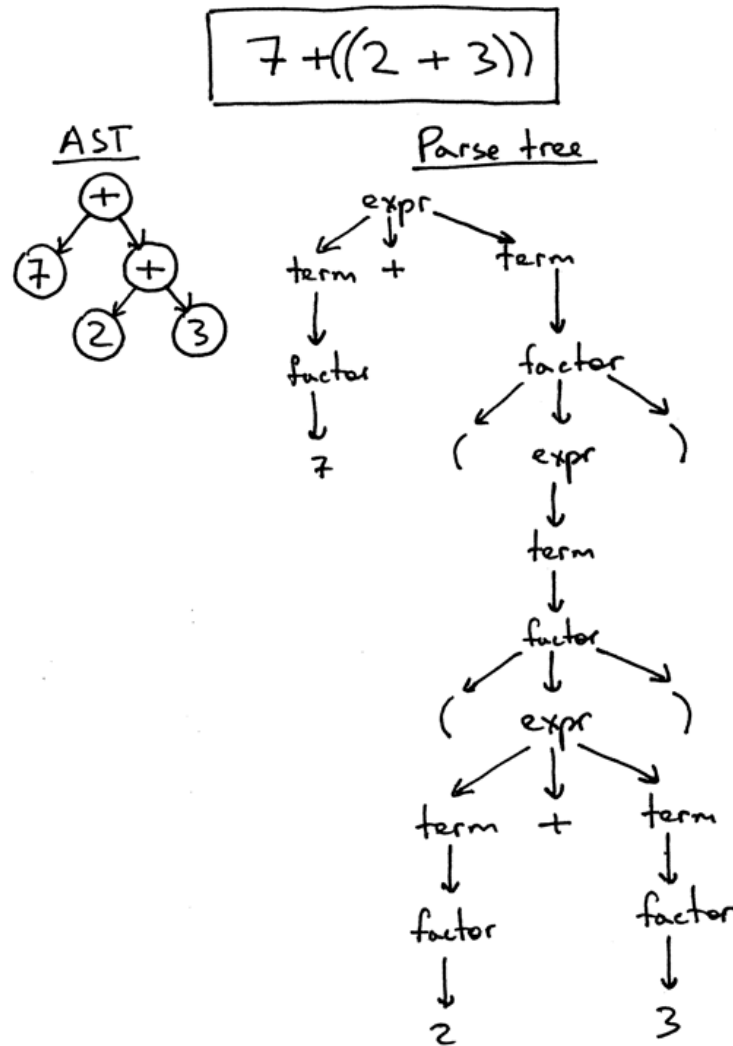
Parse Tree adalah representasi lengkap dari struktur sintaks program sesuai dengan grammar bahasa yang digunakan. Setiap node di dalam parse tree merepresentasikan **produksi grammar**, sehingga pohon ini memuat semua simbol terminal dan non-terminal.



Contoh Parse Tree

Sumber: https://textx.github.io/Arpeggio/latest/images/calc_parse_tree.dot.png

Parse tree berguna untuk menunjukkan bagaimana token-token hasil lexical analysis membentuk struktur program yang sesuai dengan aturan grammar.



Gambar Contoh Pembuatan Parse Tree dan Perbedaannya dengan AST (Abstract Syntax Tree)
(Sumber: <https://ruslanspivak.com/lsbasi-part7>)

Keterangan: pada tugas ini, diharapkan dihasilkan sebuah Parse Tree dan bukan Abstract Syntax Tree. Abstract Syntax Tree akan dibahas lebih lanjut pada milestone selanjutnya.

II. Spesifikasi Tugas Besar

Pada milestone 2, Anda diminta untuk membuat **tahapan kedua dari compiler**, yaitu **syntax analysis atau parser**. Parser berfungsi melakukan analisis sintaksis (syntax analysis) dengan menggunakan **Recursive Descent** untuk mengenali struktur gramatikal dalam deretan token.

A. Revisi Program Milestone 1

Terdapat beberapa **revisi** pada **DFA** yang dibuat sebelumnya untuk pengerjaan kedepannya, yaitu pembacaan *separator* (spasi, new-line, dan tab). Dua sekuens-karakter yang terpisah dengan *separator* **pasti** merupakan dua token yang berbeda. Sebagai contoh:

1. katasambung → ident(katasambung)
2. kata pisah → ident(kata), ident(pisah)
3. := kata → becomes, ident(kata)

Sekuens-karakter yang tidak terpisah dengan *separator* secara default dibaca berdasarkan prinsip *longest-match*. Sebagai contoh:

1. 1.23 → realcon(1.23)
2. ident:=1 → ident(ident), becomes, intcon(1)
3. .23.e-+/1.e-+/1 → *unknown*(.23.e-+/1.e-+/1)

B. Spesifikasi Program

Tahapan ini bertujuan untuk menganalisis struktur sintaksis dari list of tokens yang dihasilkan lexer dan membangun **Parse Tree** yang merepresentasikan hierarki struktur program sesuai dengan grammar bahasa Arion. Parse Tree ini akan menjadi masukan bagi tahap semantic analysis dalam proses kompilasi.

Masukan	List Token (misalnya VAR(X), ASSIGN(=), OPERATOR(+))
Keluaran	Parse Tree
Algoritma	Recursive Descent

C. Daftar Node

Berikut merupakan daftar node yang mungkin muncul pada hasil Parse Tree.

No	Nama Node	Deskripsi	Aturan Produksi	Contoh
	<program>	Node root yang merepresentasikan keseluruhan program Arion	program $\rightarrow$ program-header + declaration-part + compound-statement + period	Seluruh struktur program dari awal hingga akhir
	<program-header>	Bagian kepala program yang berisi nama program	programsy + ident + semicolon	program Hello;
	<declaration-part>	Bagian deklarasi yang dapat berisi const, type, var, procedure, function	(const-declaration)* + (type-declaration)* + (var-declaration)* + (subprogram-declaration)*	Semua deklarasi sebelum begin
	<const-declaration>	Deklarasi konstanta	constsy + (ident + eql + constant + semicolon) +	const INT == 100; REAL == 10.0; CHAR == 'a'; STRING == 'abcd';
	<constant>	Konstanta dapat berupa ident , intcon , realcon , charcon , atau string	charcon string [(plus minus)? + (ident intcon realcon)]	'a' 'abcd' 10 +10.0 -a

<type-declaration>	Deklarasi tipe data baru	typesy + (ident + eql + type + semicolon)+	type Range == 1..10; type Month == (Jan, Feb, Mar, Apr);
<var-declaration>	Deklarasi variabel	varsy + (identifier-list + colon + type + semicolon)+	var a, b: integer;
<identifier-list>	Daftar identifer yang dipisahkan koma	ident (comma + ident)*	a, b, c
<type>	Tipe data yang terdefinisi atau didefinisikan pengguna. Dapat berupa ident , array-type, range, atau enumerated	ident array-type range enumerated record-type	integer, array[1..10] of integer
<array-type>	Definisi tipe array	arraysy + lbrack + (range ident) + rbrack + ofsy + type	array[1..10] of integer array[char] of integer
<range>	Rentang nilai untuk array atau subrange	expression + period + period + expression	1..10, 'a'..'z'
<enumerated>	Nilai tertentu yang ada di dalam list	lbrack + ident + (comma + ident)* + rbrack	(Jan, Feb, Mar, Apr)

<record-type>	Tipe data template, sama dengan <i>class</i> dalam OOP	recordsy + field-list + endsy	<pre> type record-name = record field-1: field-type1; field-2: field-type2; ... field-n: field-typen; end </pre>
<field-list>	Penjabaran atribut/field dari record	field-part + (semicolon + field-part)*	<pre> ..field-1..; ..field-2.. </pre>
<field-part>	Singular atribut/field dari record	identifier-list + colon + type	<pre> field-1: field-type1; </pre>
<subprogram-declaration>	Deklarasi prosedur atau fungsi	procedure-declaration function-declaration	<pre> procedure print (x: integer); </pre>
<procedure-declaration>	Deklarasi prosedur	proceduresy + ident + (formal-parameter-list)? + semicolon + block + semicolon	Prosedur dengan parameter opsional
<function-declaration>	Deklarasi fungsi	functionsy + ident + (formal-parameter-list)? + colon + ident + semicolon + block + semicolon	Fungsi yang mengembalikan nilai

	block	Bagian kode yang mengandung deklarasi dan statement	declaration-part + compound-statement	var ... begin ... end
	<formal-parameter-list>	Daftar parameter formal	lparent + parameter-group + (semicolon + parameter-group)* + rparent	(x, y: integer; z: real)
	<parameter-group>	Daftar parameter disertai dengan tipe data parameter	identifier-list + colon + (ident array-type)	x, y: integer
	<compound-statement>	Blok statement yang diawali begin dan diakhiri end	beginsy + statement-list + endsy	begin ... end
	<statement-list>	Daftar statement yang dipisahkan semicolon	statement (semicolon + statement)*	Urutan statement dalam blok
	statement	Deskripsi aksi yang harus dilakukan program komputer	assignment-statement if-statement case-statement while-statement repeat-statement for-statement	x := 5; if x > 0 then y := 1 else y := 0 while x < 10 do x := x + 1
	<assignment-statement>	Statement penugasan nilai ke variabel	ident + becomes + expression	x := 5, y := x + 10

<if-statement>	Statement kondisional	ifsy + expression + thensy + statement + (elsy + statement)?	if x > 0 then y := 1 else y := 0
<case-statement>	Statement kondisional dengan banyak kondisional	casesy + expression + ofsy + case-block + endsy	case i of 0: x := 0; 1 : x := 1; end
<case-block>	Deklarasi case dan statement yang harus dilakukan	constant + (comma + constant)* + colon + statement + (semicolon + case-block?)*	0: x := 0; 1, 2: x := 1
<while-statement>	Perulangan dengan kondisi di awal	whilesy + expression + dosy + statement	while x < 10 do x := x + 1
<repeat-statement>	Perulangan dengan kondisi di akhir	repeatsy + statement-list + untilsy + expression	repeat x := x - 1 until x == 0
<for-statement>	Perulangan dengan counter	forsy + ident + becomes + expression + (tosy downtosy) + expression + dosy + statement	for i := 1 to 10 do ...
<procedure/function-call>	Pemanggilan prosedur atau fungsi	ident + (lparent + parameter-list + rparent)	writeln('Hello'), print(x, y)
<parameter-list>	Daftar parameter aktual saat pemanggilan	expression (comma + expression)*	'Result = ', b, 100

<expression>	Ekspresi yang menghasilkan nilai	simple-expression (relational-operator + simple-expression)?	$x + y, a > b, 5 * (x + 1)$
<simple-expression>	Ekspresi tanpa operator relasional	(plus minus)? term (additive-operator + term)*	$x + y - z, 5 * 2$
<term>	Bagian ekspresi dengan prioritas lebih tinggi	factor (multiplicative-operator + factor)*	$x * y, a \text{ bagi } b$
<factor>	Unit terkecil dalam ekspresi	ident intcon charcon string (lparent + expression + rparent) notsy + factor function-call	$x, 5, 'a', (x+y), \text{flag "tidak"}$
<relational-operator>	Operator perbandingan	eq neq gtr geq lss leq	$x == 5, y > 10$
<additive-operator>	Operator penjumlahan/pengurangan	plus minus orsy	$x + y, a \text{ atau } b$
<multiplicative-operator>	Operator perkalian/pembagian	times rdiv idiv imod andsy	$x * y, a \text{ div } b, p \text{ and } q$

Diagram syntax untuk beberapa node juga dapat dilihat pada bagian [lampiran](#).

Program harus menggunakan Recursive Descent dengan **grammar yang di-hardcode dalam program** dan keseluruhan grammar dijelaskan kembali dalam laporan yang dibuat.

Input awal dari program **tetap merupakan source code Arion** sehingga sebelum dimasukkan ke parser, kode dimasukkan terlebih dahulu ke lexer. Output syntax analyzer merupakan parse tree yang di **print ke terminal** dan **disimpan ke dalam file txt**.

Isi masukan (input) program.txt
<pre>program Hello; var a, b: integer; begin a := 5; b := a + 10; writeln('Result = ', b); end.</pre>
program.txt ketika diubah menjadi token
<pre>programsy ident(Hello) semicolon varsy ident(a) comma ident(b) colon ident(integer) semicolon beginsy ident(a) becomes intcon(5) semicolon ident(b) becomes ident(a) plus intcon(10) semicolon</pre>

```

ident(writeln)
lparent
string('Result = ')
comma
ident(b)
rparent
semicolon
endsy
period

```

Proses (grammar belum tentu benar)

```

program → program-header declaration-part compound-statement
period
program-header → programsy ident semicolon
declaration-part → ...
compound-statement → ...
... grammar lainnya

```

Berdasarkan grammar "program", non-terminal yang pertama ditemukan adalah "program-header". Setelah itu di dalam "program-header" dilakukan pengecekan token input.

```

programsy
ident
semicolon

```

Karena token input sesuai, berarti "program-header" valid. Sehingga output sementara akan seperti ini

```

<program>
├── <program-header>
│   ├── programsy
│   ├── ident(Hello)
│   └── semicolon
... sisa output

```

Kemudian sisa pengecekan dilakukan untuk "declaration-part", "compound-statement", dan period.

Kemudian juga disini dilakukan **error checking** di level parser. Contohnya adalah misal untuk "program-header", token yang

dibaca adalah seperti berikut.

```

programsy
ident(Hello)
period

```

Maka parser akan mengeluarkan error. Untuk pesan error dibebaskan yang penting informatif. Contohnya adalah "Syntax error: unexpected token period, expected semicolon".

Keluaran (output)

```

<program>
├── <program-header>
│   ├── programsy
│   ├── ident(Hello)
│   └── semicolon
├── <declaration-part>
│   └── <var-declaration>
│       ├── varsy
│       ├── <identifier-list>
│       │   ├── ident(a)
│       │   ├── comma
│       │   └── ident(b)
│       ├── colon
│       ├── <type>
│       │   └── ident(integer)
│       └── semicolon
├── <compound-statement>
│   ├── beginsy
│   ├── <statement-list>
│   │   ├── <assignment-statement>
│   │   │   ├── ident(a)
│   │   │   ├── becomes
│   │   │   └── <expression>
│   │   │       └── <simple-expression>
│   │   │           └── <term>
│   │   │               └── <factor>
│   │   │                   └── intcon(5)
│   │   ├── semicolon
│   │   └── <assignment-statement>
│   │       └── ident(b)

```



D. Laporan

Anda wajib membuat laporan yang setidaknya berisi:

- Cover dengan foto kelompok menggantikan logo ITB.
- Teori Singkat
 - Jelaskan teori-teori atau konsep yang digunakan (Parser, Parse Tree, Recursive Descent, dsb)
- Perancangan & Implementasi
 - Jelaskan implementasi program, terutama yang berhubungan dengan Parser.
- Pengujian
 - Setidaknya 5 pengujian kasus unik.
 - Setiap pengujian harus menunjukkan hasil input/output program.
 - Bukti berupa screenshot input/output program.
- Kesimpulan dan Saran
- Lampiran berupa:
 - Link Repository Github.
 - Pembagian Tugas (menunjukkan persentase kontribusi).
- Referensi

III. Teknis Pengerjaan

- Anda **melanjutkan program hasil milestone 1** dan menambahkan komponen **parser** dengan Bahasa pemrograman yang **disamakan** dengan pengerjaan milestone sebelumnya (C/C++).
- Anda dipersilahkan untuk memperbaiki kesalahan dari milestone sebelumnya karena dapat mempengaruhi hasil dari parser. Namun tidak ada penilaian ulang untuk hasil milestone sebelumnya. Jangan lupa menginvite asisten masing-masing ke dalam repository agar tugas dapat dinilai.
- Pengerjaan dilakukan pada **repository yang sama** dengan **kelompok yang sama** sesuai [sheets pembagian kelompok](#).
- **Jangan lupa menginvite asisten masing-masing ke dalam repository agar tugas dapat dinilai.**
- Repository setidaknya mengandung:
 - Folder **src** untuk menyimpan seluruh **source code**
 - Folder **doc** untuk menyimpan **laporan**
 - Folder **test** untuk menyimpan **semua input/output** yang dilakukan pada bagian pengujian laporan. Untuk membedakan dengan milestone lain, maka file dipisahkan ke dalam folder untuk setiap milestone. Sebagai contoh: ``test/milestone-2/output-1.txt``.
 - **README** yang setidaknya mengandung:
 - Identitas Kelompok
 - Deskripsi Program
 - Requirements
 - Cara Instalasi dan Penggunaan Program
 - Pembagian Tugas

- Jika ada pertanyaan lebih lanjut, silahkan ditanyakan pada [QNA](#).

IV. Pengumpulan

- **Pengumpulan bersifat Hard Deadline**
- Pada setiap Milestone, Anda **WAJIB** membuat **Release**.
 - Gunakan format tag berupa **v0.X.Y** dengan **X** adalah **nomor milestone** dan **Y** adalah **nomor pengumpulan**.
 - Sebagai contoh, untuk milestone 1, release pertama, dan pengumpulan pertama adalah **v0.1.1**. Untuk pengumpulan selanjutnya karena revisi, gunakan tag **v0.1.2** dan seterusnya sesuai nomor pengumpulan. Untuk milestone selanjutnya, pengumpulan dimulai kembali dari satu yaitu **v0.2.1, V0.2.2**, dan seterusnya.
- **Laporan** dikumpulkan dengan cara **meletakkan** hasil **PDF** dalam **folder doc** dengan format **Laporan-X-[KODE].pdf**, dimana **X** adalah **nomor milestone**. Pastikan isi laporan sudah sesuai dengan [teknis pengerjaan](#).
- Link Pengumpulan: [Pengumpulan Milestone](#)
- Deadline milestone: **Kamis, 7 Mei 2026 pukul 23.59 WIB**

V. Penilaian

Program (70 poin)			
No	Komponen	Poin	Syarat Poin Maksimal
1.	Implementasi Recursive Descent	10	Algoritma diimplementasikan secara tepat dan dapat memeriksa kebenaran syntax pada input
2.	Kebenaran Parse Tree	12	Parse Tree yang dihasilkan sesuai dengan input token yang diberikan serta grammar bahasa Arion
3.	Struktur/modularitas program	8	Kode ditulis dengan struktur modular menggunakan fungsi atau kelas yang terpisah berdasarkan tanggung jawab. Alur eksekusi program mudah diikuti dan tidak redundan
4.	Error handling	5	Program dapat menangani error dengan baik (pesan informatif dan error tidak menyebabkan crash)
5.	Penerimaan input	3	Program dapat membaca file input hasil tokenisasi pada milestone 1 (.txt)
6.	Penghasilan output	2	Program mengeluarkan output Parse Tree yang terformat dengan jelas ke terminal dan ke dalam file txt
7.	Dokumentasi/komentar	5	Kode diberi komentar yang informatif dan tidak berlebihan
8.	Aturan Grammar	10	Grammar yang tepat untuk bahasa Arion di hardcode secara terstruktur

9.	Pengujian	15	Program berhasil lolos seluruh kasus uji
----	-----------	----	--

Laporan (30 poin)			
No	Komponen	Poin	Syarat Poin Maksimal
1.	Penjelasan konsep	10	Penjelasan mengenai konsep lexer yang setidaknya mencakup: • Definisi dan peran parser • Keterkaitan dengan lexer • Algoritma Recursive Descent • Parse Tree
2.	Perancangan program	5	Penjelasan mengenai rancangan program yang mencakup teknologi yang digunakan, struktur program, fungsi/kelas utama, dan alur kerja program. Terdapat pula justifikasi atas pilihan implementasi
3.	Hasil implementasi	2	Dokumentasi mengenai hasil implementasi, dapat berupa tangkapan layar dengan penjelasan
4.	Test case	3	Dokumentasi mengenai pengujian mandiri (minimal 5 dan mencakup berbagai kasus)
5.	Aturan Grammar	10	Tuliskan kembali seluruh Grammar yang digunakan secara rapi

Referensi

“Let’s Build A Simple Interpreter. Part 7: Abstract Syntax Trees”

<https://ruslanspivak.com/lsbasi-part7>

“Contoh Gambar Parse Trees”

https://textx.github.io/Arpeggio/latest/images/calc_parse_tree.dot.png

Lampiran A. Link Penting

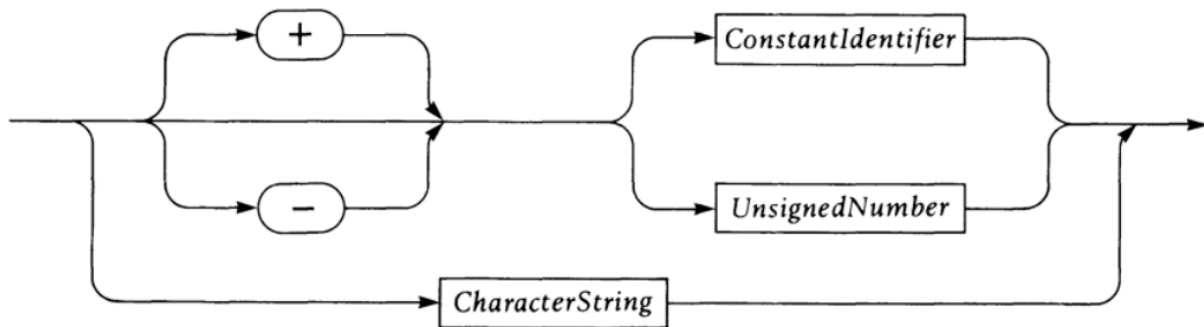
Laman Pembagian Kelompok: [📁 Pembagian Kelompok - Tubes IF2224 TBFO](#)

Laman QnA Tugas Besar: [📁 QNA - Tubes IF2224 TBFO](#)

From Pengumpulan: [Form Pengumpulan - Tubes IF2224 TBFO](#)

Lampiran B. Syntax Diagram

Constant



CaseStatement

